# AGILE SCRUM METHODOLOGY

Comprehensive Examination Answer Book

All 54 Questions – Detailed Answers

# UNIT 1: Introduction to Agile & Scrum

## Q1. Define Agile Scrum

### Definition:

Agile Scrum is an iterative and incremental project management framework used primarily in software development. It is a subset of the Agile methodology that provides a structured yet flexible approach to delivering complex products. Scrum organizes work into fixed-length cycles called Sprints (typically 1–4 weeks), during which a cross-functional team collaborates to deliver a potentially shippable product increment.

### Key Characteristics:

- Empirical Process Control: Scrum is based on transparency, inspection, and adaptation.
- Time-boxed Sprints: Each Sprint is a fixed-length iteration (usually 2 weeks).
- Defined Roles: Product Owner, Scrum Master, and Development Team.
- Defined Events: Sprint Planning, Daily Scrum, Sprint Review, Sprint Retrospective.
- Defined Artifacts: Product Backlog, Sprint Backlog, and Increment.

### Agile Manifesto Values (Foundation of Scrum):

- Individuals and interactions over processes and tools.
- Working software over comprehensive documentation.
- Customer collaboration over contract negotiation.
- Responding to change over following a plan.

### Purpose:

Scrum helps teams deliver value faster by breaking work into manageable pieces, continuously inspecting outcomes, and adapting based on feedback. It reduces risk by ensuring frequent delivery and early detection of issues.

## Q2. Explain Incremental Approach versus Agile Development

### Incremental Approach:

In the incremental model, the software is built and delivered in parts (increments). Each increment adds new functionality to the existing system. Requirements are usually defined at the beginning, but delivery happens in stages. The system grows with each release.

### Characteristics of Incremental Approach:

- Requirements are fixed at the start.
- Software is divided into modules or builds.
- Each increment delivers a working piece of the system.
- Testing is done after each increment.
- Risk is reduced compared to Waterfall because early increments can be tested.

### Agile Development:

Agile is a philosophy and set of principles that promotes iterative development, collaboration, and customer feedback. In Agile, requirements and solutions evolve through the collaborative effort of cross-functional teams. Agile welcomes changing requirements even late in development.

### Characteristics of Agile:

- Iterative and incremental: work is done in short cycles (iterations/sprints).
- Highly flexible: requirements can change between iterations.
- Customer involvement throughout the project.
- Continuous delivery of working software.
- Focus on collaboration, communication, and adaptation.

### Key Differences:

| Parameter | Incremental | Agile |
|---|---|---|
| Requirements | Fixed at start | Evolving throughout |
| Customer Involvement | Limited | Continuous |
| Flexibility | Low | High |
| Delivery | End of each increment | End of each sprint |
| Change handling | Difficult | Welcomed |
| Team collaboration | Moderate | High |

## Q3. Define Sprint in Agile

### Definition of Sprint:

A Sprint (also called an Iteration) is a fixed-length, time-boxed period during which a Scrum team works to complete a set of pre-selected Product Backlog Items (PBIs) and deliver a potentially shippable product Increment. Sprints are the heartbeat of Scrum, where ideas are turned into value.

### Key Properties of a Sprint:

- Duration: Usually 1 to 4 weeks (most commonly 2 weeks). The duration is fixed and does not change during the sprint.
- Sprint Goal: A single objective that gives the team a unified purpose for the Sprint.
- No changes: During a Sprint, no changes are made that endanger the Sprint Goal.
- Quality standards: Quality does not decrease during a Sprint.
- Sprint Backlog: The set of Product Backlog items selected for the Sprint plus the plan for delivering them.

### Events Within a Sprint:

1. Sprint Planning – Define what to do and how to do it.
2. Daily Scrum – 15-minute daily sync meeting.
3. Sprint Review – Inspect the Increment and adapt the Product Backlog.
4. Sprint Retrospective – Reflect on the process and improve.

### Why Sprints are Important:

- They create a regular cadence of delivery, reducing risk.
- They enable early feedback from stakeholders.
- They promote accountability and transparency within the team.

## Q4. Explain Lean Principles of Agile

### Overview:

Lean thinking originated from Toyota's Production System and was adapted into software development. Lean principles form a key philosophical foundation of the Agile movement. There are 7 core Lean principles:

### 1. Eliminate Waste:

Waste is anything that does not add value to the customer. In software, waste includes: unnecessary code or features, unclear requirements, waiting, task switching, defects, and over-processing. The goal is to identify and eliminate all forms of waste.

### 2. Build Quality In:

Quality should be built into the process, not inspected in at the end. Techniques like Test-Driven Development (TDD), automated testing, and peer reviews help catch defects early.

### 3. Create Knowledge:

Teams should invest in learning and documentation. Knowledge gained during development should be shared and codified. This prevents rework and promotes continuous improvement.

### 4. Defer Commitment:

Decisions should be delayed until the last responsible moment. This allows teams to gather more information before committing to a solution, reducing the risk of making costly decisions too early.

### 5. Deliver Fast:

The faster you can deliver value, the sooner you get feedback. Speed enables faster learning cycles. Short iterations (Sprints) in Scrum are aligned with this principle.

### 6. Respect People:

People are the most important asset. Empower team members, trust them, and give them the information and authority to make decisions. A motivated team is a productive team.

### 7. Optimize the Whole:

Do not just optimize one part of the system. Optimize the entire value stream from idea to delivery. Avoid local optimizations that hurt global performance.

## Q5. Define Transparency, Inspection, and Adaptation with respect to Empirical Process Control Model

### Empirical Process Control Model:

Scrum is founded on empirical process theory (empiricism), which asserts that knowledge comes from experience and decisions should be made based on what is known. Scrum uses three pillars to uphold empirical process control:

### 1. Transparency:

All aspects of the process must be visible to those responsible for the outcome. Significant aspects of the process must be defined by a common standard so that observers share a common understanding.

- Example: The Sprint Backlog is visible to the entire team so everyone knows what is being worked on.
- The Definition of Done (DoD) is transparent so all team members agree on what 'complete' means.
- The Product Backlog is transparent to all stakeholders.

### 2. Inspection:

Scrum users must frequently inspect Scrum artifacts and progress toward a Sprint Goal to detect undesirable variances. Inspection should not be so frequent that it gets in the way of work.

- Example: The Daily Scrum is an inspection of progress toward the Sprint Goal.
- The Sprint Review inspects the Increment.
- The Sprint Retrospective inspects the process itself.

### 3. Adaptation:

If an inspector determines that one or more aspects of a process deviate outside acceptable limits, the process or the material being processed must be adjusted. Adjustment must be made as soon as possible to minimize further deviation.

- Example: If Daily Scrum reveals a blocker, the team adapts their plan.
- Sprint Retrospective leads to adaptations in the team's working process.

These three pillars are interdependent. Without transparency, inspection is meaningless. Without inspection, adaptation cannot happen correctly.

## Q6. Elaborate Key Roles: i) Product Owner ii) Scrum Master iii) Developer

### i) Product Owner (PO):

The Product Owner is responsible for maximizing the value of the product resulting from the work of the Scrum Team. The PO is the sole person responsible for managing the Product Backlog.

Key Responsibilities:

- Developing and explicitly communicating the Product Goal.
- Creating and clearly communicating Product Backlog items (User Stories).
- Ordering (prioritizing) the Product Backlog to best achieve goals.
- Ensuring the Product Backlog is transparent, visible, and understood.
- Acting as the voice of the customer and stakeholders.
- Accepting or rejecting work done by the development team.

### ii) Scrum Master (SM):

The Scrum Master is responsible for establishing Scrum as defined in the Scrum Guide. They help everyone understand Scrum theory, practices, rules, and values. The Scrum Master is a servant-leader for the Scrum Team.

Key Responsibilities:

- Coaching the team in self-management and cross-functionality.
- Helping the Scrum Team focus on creating high-value Increments.
- Removing impediments (blockers) to the team's progress.
- Ensuring Scrum events take place and are positive and productive.
- Facilitating stakeholder collaboration.
- Protecting the team from external interruptions.

### iii) Developer (Development Team):

Developers are the people in the Scrum Team committed to creating any aspect of a usable Increment each Sprint. They are cross-functional and self-organizing.

Key Responsibilities:

- Creating a plan for the Sprint — the Sprint Backlog.

- Instilling quality by adhering to a Definition of Done.
- Adapting their plan each day toward the Sprint Goal.
- Holding each other accountable as professionals.
- Estimating and committing to work during Sprint Planning.
- Participating in Daily Scrum, Sprint Review, and Sprint Retrospective.

## Q7. Discuss Scrum Events: i) Sprint ii) Sprint Planning iii) Daily Scrum iv) Sprint Retrospective

### i) Sprint:

The Sprint is the core event — a fixed-length container for all other Scrum events. Duration: 1–4 weeks. Each Sprint starts immediately after the conclusion of the previous one. During a Sprint, no changes are made that would endanger the Sprint Goal, quality doesn't decrease, and the Product Backlog is refined as needed. A Sprint can be cancelled by the Product Owner if the Sprint Goal becomes obsolete.

### ii) Sprint Planning:

Sprint Planning initiates the Sprint by laying out the work to be performed. It is time-boxed to a maximum of 8 hours for a one-month Sprint.

Three topics addressed in Sprint Planning:

5. Why is this Sprint valuable? – The PO proposes the Sprint Goal.
6. What can be Done this Sprint? – Team selects items from Product Backlog.
7. How will the chosen work get done? – Team decomposes work into tasks of 1 day or less.

### iii) Daily Scrum:

The Daily Scrum is a 15-minute event held each day of the Sprint for the Developers. It inspects progress toward the Sprint Goal and adapts the Sprint Backlog as necessary.

Classic three questions (though format can vary):

- What did I do yesterday that helped the team meet the Sprint Goal?
- What will I do today to help the team meet the Sprint Goal?
- Do I see any impediments preventing me or the team from meeting the Sprint Goal?

Benefits: Improves communication, identifies impediments early, promotes quick decision-making, and eliminates the need for other meetings.

### iv) Sprint Retrospective:

The Sprint Retrospective is the last event of the Sprint (before the next Sprint Planning). It is time-boxed to 3 hours for a one-month Sprint. Its purpose is to plan ways to increase quality and effectiveness.

The team inspects:

- How the last Sprint went with regards to individuals, interactions, processes, tools, and Definition of Done.
- What went well during the Sprint?
- What problems were encountered?
- How were those problems solved (or not)?

Output: Actionable improvements added to the next Sprint Backlog.

## Q8. Describe how Increment Guarantees Definition of Done (DoD)

### What is an Increment?

An Increment is a concrete stepping stone toward the Product Goal. Each Increment is additive to all prior Increments and thoroughly verified, ensuring that all Increments work together. In order to provide value, the Increment must be usable.

### What is the Definition of Done (DoD)?

The Definition of Done is a formal description of the state of the Increment when it meets the quality measures required for the product. The moment a Product Backlog item meets the Definition of Done, an Increment is born.

### How DoD Guarantees Quality:

- The DoD creates transparency. Everyone knows what 'done' means, preventing misunderstandings.
- It sets a minimum quality standard. Work that doesn't meet DoD is not included in an Increment.
- It ensures the Increment is potentially releasable after every Sprint.
- It builds trust with stakeholders who know the team's quality standard.

### Example DoD for a Software Team:

- Code is written and peer-reviewed.
- Unit tests written and passing (>80% coverage).
- Integration tests passing.
- No critical/high bugs open.
- Documentation updated.
- Deployed to staging environment.
- Accepted by Product Owner.

### Key Rule:

If a Product Backlog item does not meet the Definition of Done, it cannot be included in the Increment. It is returned to the Product Backlog for future consideration. The DoD is the gatekeeper of quality and the guarantor of a reliable Increment.

## Q9. Explain Product Management with respect to Growth, Maturity, and Decline with suitable example

### Product Life Cycle (PLC) in Agile Product Management:

Product Management involves strategically managing a product from its inception to its retirement. The Product Life Cycle consists of four stages: Introduction, Growth, Maturity, and Decline. Agile product management uses Scrum to adapt the backlog and strategy at each stage.

### 1. Introduction Stage:

The product is launched. Focus is on building core features, acquiring early adopters, and gathering feedback. Sprints focus on MVP (Minimum Viable Product) features.

### 2. Growth Stage:

Sales and users increase rapidly. The Product Owner focuses the backlog on: scaling features, performance improvements, expanding to new markets, and onboarding new users.

Example (E-commerce App): In growth, the team adds payment gateways, mobile apps, referral programs, and multiple language support.

### 3. Maturity Stage:

The product reaches peak market penetration. Growth slows. The team focuses on: retaining existing users, optimizing performance, competitive differentiation, and reducing technical debt.

Example (E-commerce App): Sprints focus on personalization algorithms, loyalty programs, advanced analytics, and UI/UX improvements.

### 4. Decline Stage:

User numbers and revenue start falling, often due to better competing products. The organization must decide: revive the product (pivot), sunset it, or maintain it with minimal investment.

Example (E-commerce App): If a new competitor offers better technology, the team may pivot to a subscription model or wind down operations.

### Agile's Role Across the PLC:

Scrum allows the Product Owner to continuously reprioritize the backlog based on the product's stage, market feedback, and business strategy, ensuring the team always works on the most valuable items at any given lifecycle phase.

# UNIT 2: Model Selection & Scenario-Based Questions

## Q10. Gaming company – continuous improvement based on player feedback. Which model?

### Recommended Model: Agile Scrum (Iterative & Incremental)

### Justification:

A gaming company that wants to continuously improve graphics, gameplay, and features based on player feedback is a perfect use case for the Agile Scrum model. Here is why:

- Continuous Improvement: Scrum's Sprint-based cycles allow the team to release a new game version at the end of each sprint (e.g., every 2 weeks) and then incorporate player feedback into the next sprint.
- Changing Requirements: Players' preferences evolve constantly. Agile welcomes change even late in development. The Product Backlog is continuously refined based on what players want.
- Iterative Delivery: Each sprint delivers an improved version of the game — better graphics, new levels, bug fixes — without waiting for the entire game to be redesigned.
- Player Feedback Loop: Sprint Reviews with stakeholders (game testers, players) allow the team to inspect the game and adapt their plan for the next sprint.
- Cross-Functional Teams: Game development needs programmers, designers, artists, and testers — all of which can be part of a Scrum team.

### Example Sprint Plan:

- Sprint 1: Improve frame rate and graphics engine.
- Sprint 2: Add two new gameplay mechanics based on feedback.
- Sprint 3: Fix reported bugs and optimize loading times.

### Conclusion:

Agile Scrum is the best fit because it supports rapid release cycles, accommodates player-driven changes, and promotes a culture of continuous improvement — all critical for a gaming product.

## Q11. Banking application – security first, then analytics. Fixed requirements. Which model?

### Recommended Model: Incremental Model (or V-Model for safety-critical parts)

### Justification:

A banking application where security and core features must be completed first, followed by additional features, with mostly fixed requirements, is best suited for the Incremental Development Model.

- Fixed Requirements: The incremental model works well when requirements are clear and stable at the beginning — ideal for banking systems where regulatory and security requirements are well-defined.
- Priority-Based Delivery: The model allows delivering the most critical functionality first (security, authentication, core banking) and then adding features incrementally (analytics, notifications).
- Risk Management: Banking systems cannot afford partial security. Delivering core features first ensures a secure foundation before building analytics on top.
- Auditability: Banks require thorough documentation and testing at each increment, which the incremental model supports.

### Delivery Plan:

| Increment | Features Delivered |
|---|---|
| Increment 1 | User Authentication, Security Layer, Core Banking Transactions |

| Increment | Features Delivered |
|---|---|
| Increment 2 | Account Management, Fund Transfers |
| Increment 3 | Analytics Dashboard, Reports |
| Increment 4 | Push Notifications, Alerts |

### Conclusion:

Incremental Model is best because requirements are fixed, critical features need to be prioritized and delivered first, and each increment undergoes thorough testing before the next begins — ensuring stability and security.

## Q12. Startup AI app – unsure of user interaction, release prototypes step by step. Which model?

### Recommended Model: Agile with Prototyping / Iterative Model (Scrum)

### Justification:

A startup that is uncertain about user interaction and wants to release prototypes iteratively is an ideal fit for Agile Scrum combined with Prototyping.

- Uncertainty Management: When the team is unsure how users will interact with the AI app, iterative prototyping allows them to test hypotheses and learn from real user behavior.
- Evolving Requirements: AI-based apps evolve based on model performance and user feedback. Agile allows requirements to be refined after each sprint based on what the prototype reveals.
- Fail Fast, Learn Fast: Releasing a basic prototype early allows the startup to validate or invalidate their assumptions before investing heavily in a specific direction.
- MVP Approach: Agile promotes building a Minimum Viable Product first, then enhancing it — perfect for a startup with limited resources.
- Sprint-Driven Prototypes: Each sprint produces a testable prototype that can be shown to users, gathering real-world data on how they interact with the AI features.

### Example Sprint Workflow:

- Sprint 1: Basic AI chatbot prototype with 5 test users.
- Sprint 2: Refine UI based on user interaction data; add intent recognition.
- Sprint 3: Add personalization based on usage patterns.

### Conclusion:

Agile Scrum with Prototyping is most appropriate because the startup needs to experiment quickly, validate ideas, and pivot based on user feedback — which is exactly what iterative sprints enable.

## Q13. Payroll system – clearly defined requirements, delivered module by module. Which model?

### Recommended Model: Incremental Model

### Justification:

A payroll system where all requirements are clearly defined in advance and the system is to be delivered module by module is best suited for the Incremental Development Model.

- Clear Requirements: In a payroll system, requirements such as salary calculation rules, tax slabs, and report formats are defined by law or HR policy — they do not change frequently. The incremental model is ideal when requirements are stable.
- Modular Delivery: The system can be broken into independent modules (Salary, Tax, Reports), each delivered as a separate increment. Each module is fully functional and tested before the next begins.
- Early Usability: HR can start using the Salary module even while the Tax module is being developed, providing immediate value.
- Reduced Integration Risk: Since modules are integrated one at a time, integration problems are identified early and are isolated.

### Delivery Increments:

| Increment | Module | Description |
|---|---|---|
| 1 | Salary Module | Basic salary computation, pay slips, and bank transfer integration |
| 2 | Tax Module | Income tax calculation, TDS deductions, and Form 16 generation |
| 3 | Reports Module | Monthly payroll reports, year-end summaries, and compliance reports |

### Conclusion:

The Incremental Model is the best choice because requirements are fully known upfront, the system naturally breaks into independent modules, and the organization benefits from early delivery of critical payroll functionality.

# UNIT 3: Agile Teams & Self-Organization

## Q14. Interpret the Assembling Team during Adoption with respect to Team Inertia & Personality Type

### Team Inertia:

Team inertia refers to the resistance a team shows toward change when adopting Agile/Scrum. Teams that have worked in traditional (Waterfall) environments for a long time tend to resist the cultural shift required by Agile. This inertia manifests as:

- Reluctance to attend Daily Scrums ('we're too busy for meetings').
- Clinging to old reporting structures and hierarchical decision-making.
- Resistance to cross-functional collaboration ('that's not my job').
- Fear of transparency and public accountability.

### Overcoming Team Inertia:

- The Scrum Master must coach and guide the team patiently.
- Leadership must demonstrate commitment to Agile values.
- Small early wins in sprints help build confidence and buy-in.
- Retrospectives are used to surface resistance and address it openly.

### Personality Types and Agile Adoption:

Different personality types respond differently to Agile adoption:

| Personality Type | Behavior in Agile Adoption | Approach |
|---|---|---|
| Innovators | Embrace Agile enthusiastically, drive change | Give them leadership roles in adoption |
| Early Adopters | Open to change, quick learners | Use as champions to influence others |
| Early Majority | Accept change when they see it working | Show them success stories |
| Late Majority | Skeptical, need proof before adopting | Provide data, pair with early adopters |
| Laggards | Strongly resist change | Coach heavily or reassign roles |

Understanding these personality types allows the Scrum Master to tailor coaching strategies for each team member during Agile adoption.

## Q15. Describe Autonomy and Decision Making in Self-Organization in Team

### What is a Self-Organizing Team?

A self-organizing team in Scrum is one that chooses how best to accomplish their work, rather than being directed by others outside the team. Self-organization does not mean chaos — it means the team has the authority and trust to make decisions within the boundaries set by the organization and the Product Owner.

### Autonomy in Self-Organizing Teams:

- Task Autonomy: Team members choose which tasks to pick up from the Sprint Backlog rather than being assigned work by a manager.
- Technical Autonomy: The team decides the technical approach, tools, and architecture to use for delivering the Sprint Goal.

- Process Autonomy: The team decides their own working agreements — when they do code reviews, how they run the Daily Scrum, etc.

### Decision Making in Self-Organizing Teams:

- Decisions about 'how to build' rest with the Developers.
- Decisions about 'what to build' rest with the Product Owner.
- Decisions about 'how to improve the process' are made collectively in the Sprint Retrospective.
- The Scrum Master removes impediments to ensure the team can make and execute decisions without delays.

### Benefits of Self-Organization:

- Higher motivation and ownership — people are more committed to decisions they make themselves.
- Better quality — team members take pride in their work.
- Faster decision-making — no waiting for management approval on technical decisions.
- Increased innovation — team members feel empowered to try new approaches.

## Q16. Summarize the Importance of T-Shaped Team

### What is a T-Shaped Professional?

A T-shaped person has deep expertise in one skill (the vertical bar of the 'T') and broad knowledge across multiple disciplines (the horizontal bar). In a T-shaped team, all members possess this dual competency.

### Diagram of T-Shape:

Horizontal: Broad knowledge in design, testing, deployment, documentation, etc.

Vertical: Deep expertise in one area (e.g., backend development, UX design, data analysis).

### Importance of T-Shaped Team in Agile:

- Flexibility: T-shaped members can help in areas outside their core expertise when bottlenecks occur. If the tester is sick, a developer with testing knowledge can step in.
- Cross-Functional Collaboration: Team members understand each other's work deeply enough to collaborate effectively and give meaningful feedback.
- Reduced Bottlenecks: In traditional teams, work piles up waiting for a specialist. T-shaped teams reduce these bottlenecks because multiple people can handle adjacent tasks.
- Better Estimation: Members with broad knowledge can more accurately estimate work across different areas.
- Knowledge Sharing: T-shaped people share their deep expertise with others, continuously broadening the team's overall skill base.
- Higher Team Velocity: Because work is not blocked waiting for the single specialist, the team delivers more per sprint.

### Example:

A Backend Developer (deep expertise) who also understands: frontend basics, database queries, CI/CD pipelines, and unit testing — can contribute to multiple areas of a sprint and unblock colleagues.

## Q17. Explain the Characteristics of Product Backlog

### What is a Product Backlog?

The Product Backlog is an ordered list of everything that is known to be needed in the product. It is the single source of work undertaken by the Scrum Team. It is owned and managed by the Product Owner. The Product Backlog is never complete — it evolves as the product and the environment in which it will be used evolves.

### Key Characteristics (DEEP):

#### 1. Detailed Appropriately (D):

Items at the top of the backlog (to be worked on soon) are more detailed, refined, and estimated. Items lower in the backlog are less detailed because they are further from being worked on.

#### 2. Estimated (E):

Each item in the Product Backlog has an estimate of the effort required to complete it (in story points or hours). Estimates help in planning and forecasting.

#### 3. Emergent (E):

The Product Backlog is never finished. As the product evolves and new requirements emerge (from users, market changes, or technology), new items are added and existing ones are updated or removed.

#### 4. Prioritized (P):

Items are ordered by the Product Owner based on value, risk, and dependencies. Higher-priority items are at the top and worked on first.

### Additional Characteristics:

- Single Source of Truth: All work for the product comes from the Product Backlog.
- Transparent: The entire Scrum Team and stakeholders can see the backlog.
- Refined Continuously (Backlog Grooming): The team regularly reviews, updates, and estimates backlog items to keep it ready for Sprint Planning.

## Q18. Justify the statement: 'A Product Backlog needs prioritization'

### Statement Justification:

Prioritization of the Product Backlog is one of the most critical responsibilities of the Product Owner. Here is why the Product Backlog must always be prioritized:

#### 1. Not Everything Can Be Done First:

A product has hundreds of features, bug fixes, and improvements. The team can only work on a limited number of items per sprint. Without prioritization, the team may work on low-value features while high-value features wait.

#### 2. Maximizing Business Value:

Prioritization ensures that the most valuable features (those that generate the most revenue, solve the biggest user pain points, or meet critical business needs) are delivered first. This maximizes ROI on development investment.

#### 3. Managing Risk:

High-risk or technically challenging items should be prioritized early so that risks are identified and managed sooner, rather than discovering critical problems at the end of the project.

#### 4. Stakeholder Alignment:

Prioritization forces stakeholders to agree on what is most important, creating alignment and reducing scope creep. It also manages stakeholder expectations clearly.

### 5. Resource Optimization:

With a prioritized backlog, the team always knows what to work on next if they finish early, avoiding idle time and ensuring continuous flow of value.

### Prioritization Techniques:

- MoSCoW Method (Must Have, Should Have, Could Have, Won't Have)
- Weighted Shortest Job First (WSJF)
- Kano Model (Customer satisfaction vs. feature presence)
- Dot Voting by stakeholders

## Q19. Explain Cross-Functional Team in Agile and its Importance

### What is a Cross-Functional Team?

A cross-functional team in Agile is a team that has all the skills and competencies necessary to deliver a working product increment without depending on external people. The team collectively possesses skills in design, development, testing, deployment, and any other area needed to deliver the product.

### Characteristics of a Cross-Functional Team:

- Self-sufficient: Can complete work end-to-end without external dependencies.
- Diverse skills: Members have different areas of expertise (frontend, backend, QA, DevOps, UX).
- Shared ownership: The entire team owns the Sprint Goal and the product quality.
- Collaborative: Members work together, not in silos.

### Importance in Agile:

- Eliminates Handoffs: In traditional teams, work passes from one specialist to another (developer → tester → DevOps), creating delays. Cross-functional teams eliminate these handoffs.
- Faster Delivery: Everything needed to build and test a feature exists within the team, enabling faster sprint completion.
- Better Problem Solving: Diverse perspectives lead to more creative and robust solutions.
- Reduced Risk: If one team member is unavailable, others with overlapping skills can step in.
- Accountability: The team, not individuals, is responsible for delivering the Sprint Goal, fostering collective accountability.

### Example:

For a Sprint Goal of 'Enable users to pay via UPI,' a cross-functional team would include: a Backend Developer (payment API), a Frontend Developer (UI/UX), a QA Engineer (testing payment flows), a DevOps Engineer (deployment), and a Business Analyst (requirements).

## Q20. How can you trace Sprint Backlog using Estimation with Story Point and Estimation with Kanban?

### Tracing Sprint Backlog with Story Points:

Story points are a unit of measure for expressing the overall effort required to fully implement a Product Backlog Item. They are relative, not time-based.

- During Sprint Planning, each backlog item is estimated in story points using techniques like Planning Poker, Fibonacci sequence (1, 2, 3, 5, 8, 13, 21...).
- The total story points planned for the sprint become the Sprint capacity.

- Each day, the team tracks: Story points completed vs. story points remaining.
- This data feeds the Sprint Burndown Chart, showing actual progress vs. ideal burndown.
- Velocity = Story points completed per sprint. This helps forecast future sprints.

### Example Trace:

| Backlog Item | Story Points | Status |
|---|---|---|
| Login Feature | 5 | Done |
| User Profile Page | 8 | In Progress |
| Password Reset | 3 | To Do |
| Email Notification | 5 | To Do |

### Tracing Sprint Backlog with Kanban:

Kanban is a visual workflow management method. A Kanban board has columns representing workflow stages:

- To Do → In Progress → In Review → Done

Tracing with Kanban:

- Each backlog item is represented as a card on the board.
- Cards move across columns as work progresses.
- Work-In-Progress (WIP) limits are set for each column to prevent overloading.
- Lead Time: Time from card creation to completion — tracks delivery speed.
- Cycle Time: Time from when work starts to when it's done — tracks team efficiency.
- Cumulative Flow Diagram: Shows how many items are in each stage over time, revealing bottlenecks.

## Q21. Explain Spike for New Technology with example (complex requirement, proof of concept)

### What is a Spike?

A Spike is a special type of user story or task in Agile/Scrum used to research, investigate, or explore a new technology, approach, or solution. The goal of a spike is to gain the knowledge needed to reduce uncertainty and risk before committing to building a feature.

### When is a Spike Used?

- When the team encounters a complex, unfamiliar requirement.
- When a new technology needs to be evaluated before committing to it.
- When the team needs to create a Proof of Concept (PoC) to validate feasibility.
- When estimation is impossible without more research.

### Types of Spikes:

- Technical Spike: Researching a technical solution (e.g., 'Can we integrate Payment Gateway X with our system?').
- Functional Spike: Understanding a complex business requirement (e.g., 'What exactly does GDPR compliance require of our data storage?').

### Example:

Scenario: An e-commerce company wants to add AI-based product recommendations.

Complex Requirement: Integrate a machine learning model to recommend products to users in real-time based on browsing history.

Spike Task: Spend 3 days investigating: Which ML frameworks are compatible (TensorFlow vs. scikit-learn)? What is the API response time for real-time recommendations? How much infrastructure is needed?

Proof of Concept: Build a small PoC that takes 5 sample user profiles and returns top 3 product recommendations using TensorFlow.

Outcome: The PoC proves TensorFlow works within latency requirements. The team can now estimate the full feature and add it to the Product Backlog with confidence.

### Key Characteristics of a Spike:

- Time-boxed: Has a fixed time limit (e.g., 2 days, 1 sprint).
- Output is knowledge, not working code (though a PoC may be produced).
- Results are shared with the team to inform future planning.

## Q22. Explain 3-5-3 Rule of Project Organization in Detail

### What is the 3-5-3 Rule?

The 3-5-3 rule is a structural framework in Scrum that defines the number of roles, events, and artifacts in the Scrum framework. It helps teams and organizations remember the core components of Scrum.

### 3 Roles:

8. Product Owner (PO): Maximizes value; manages the Product Backlog; prioritizes features; represents stakeholders.
9. Scrum Master (SM): Facilitates Scrum; removes impediments; coaches the team; ensures Scrum values are upheld.
10. Developers: Cross-functional team that builds the product; self-organizing; responsible for delivering the Sprint Increment.

### 5 Events:

11. Sprint: The main container event (1–4 weeks).
12. Sprint Planning: Defines the Sprint Goal and selects backlog items.
13. Daily Scrum: 15-minute daily sync for the Developers.
14. Sprint Review: Inspect the Increment and adapt the Product Backlog.
15. Sprint Retrospective: Reflect on the process and identify improvements.

### 3 Artifacts:

16. Product Backlog: Ordered list of all desired product work.
17. Sprint Backlog: Items selected for the Sprint + plan for delivering them.
18. Increment: The sum of all completed Product Backlog items, meeting the Definition of Done.

### Commitments for Each Artifact:

| Artifact | Commitment |
|---|---|
| Product Backlog | Product Goal |
| Sprint Backlog | Sprint Goal |
| Increment | Definition of Done (DoD) |

The 3-5-3 framework ensures Scrum remains lightweight and complete — with only the essential elements needed to implement empirical process control.

## Q23. Elaborate how Scrum of Scrums delivers complex projects

### What is Scrum of Scrums (SoS)?

Scrum of Scrums is a scaling technique that allows multiple Scrum teams working on the same product to synchronize their work, coordinate dependencies, and resolve cross-team impediments. It is essentially a higher-level Scrum meeting for representatives from multiple teams.

### When is Scrum of Scrums Used?

When a product is too large for a single Scrum team (recommended: 5–9 members) and requires multiple teams (e.g., 3 teams of 7 = 21 people) working in parallel on different parts of the product.

### How Scrum of Scrums Works:

- Each individual Scrum team runs their own Sprint events (Daily Scrum, Sprint Planning, etc.).
- An ambassador or representative from each team (usually a senior developer or Scrum Master) attends the Scrum of Scrums meeting.
- The SoS meeting is held 2–3 times per week and focuses on: What has my team done since last meeting? What will my team do before the next meeting? Are there any impediments or cross-team dependencies blocking progress?

### Structure for a Complex Project Example:

Project: Build a full-featured e-commerce platform with 3 teams:

- Team A: Product Catalog and Search
- Team B: Shopping Cart and Payment
- Team C: User Account and Order Management

The SoS ensures Team B knows when Team A's search API is ready (dependency), Team C knows when Team B's order object structure is finalized, and cross-cutting concerns like authentication and logging are coordinated.

### Benefits of Scrum of Scrums:

- Enables scaling Agile to large teams without losing agility.
- Identifies and resolves cross-team impediments quickly.
- Maintains transparency across all teams.
- Preserves the self-organizing nature of individual Scrum teams.

## Q24. Explain any two Scrum estimation techniques

### 1. Planning Poker:

Planning Poker (also called Scrum Poker) is a consensus-based estimation technique used to estimate the effort or relative size of user stories. Each team member uses a deck of cards with Fibonacci numbers (0, 1, 2, 3, 5, 8, 13, 21, 40, 100).

Process:

19. The Product Owner reads a user story.
20. Each developer privately selects a card representing their estimate.
21. All cards are revealed simultaneously.

22. If there is agreement, that estimate is used.
23. If there is disagreement, the highest and lowest estimators explain their reasoning.
24. The team re-estimates until consensus is reached.

Advantage: Prevents anchoring bias (no one is influenced by others' estimates). Promotes discussion and shared understanding.

### 2. T-Shirt Sizing:

T-Shirt sizing uses clothing sizes (XS, S, M, L, XL, XXL) to estimate the relative size of user stories. It is a quick, informal estimation technique used especially in early backlog grooming when items are not well-defined.

Process:

25. Team reviews the user story.
26. Each member assigns a T-shirt size (e.g., 'This is a Medium').
27. Sizes are discussed and a consensus size is agreed upon.
28. Later, sizes are mapped to story points (e.g., S=3, M=5, L=8, XL=13).

Advantage: Fast and easy. Works well for high-level roadmap planning when detailed estimation is premature.

## Q25. Explain Dot Voting, T-Shirt Size, and 3-Point Method of effort estimation

### 1. Dot Voting:

Dot Voting is a democratic estimation/prioritization technique where team members use physical or virtual dots to vote on items.

Process: Each participant is given a fixed number of dots (e.g., 5). They distribute their dots among the backlog items based on priority or effort. Items with the most dots are prioritized first.

Use Case: More commonly used for prioritization than effort estimation. Useful in backlog refinement sessions to quickly surface consensus.

### 2. T-Shirt Size:

As described in Q24, T-Shirt sizing assigns relative size labels (XS/S/M/L/XL/XXL) to user stories.

Mapping Example:

| T-Shirt Size | Story Points | Approx. Effort |
|---|---|---|
| XS | 1 | < 1 day |
| S | 2 | 1 day |
| M | 5 | 2–3 days |
| L | 8 | 3–5 days |
| XL | 13 | 1 week |
| XXL | 21+ | Needs breakdown |

### 3. Three-Point Method (PERT-based):

The Three-Point Method estimates effort using three values to account for uncertainty:

- Optimistic (O): Best-case effort if everything goes well.
- Pessimistic (P): Worst-case effort if things go wrong.

- Most Likely (M): Most realistic expected effort.

Formula: Expected Effort (E) = (O + 4M + P) / 6

Example: A login feature. O = 2 days, M = 4 days, P = 8 days.

E = (2 + 4×4 + 8) / 6 = (2 + 16 + 8) / 6 = 26 / 6 ≈ 4.33 days

This gives a more realistic estimate than a single-point guess.

## Q26. Elaborate the estimation of Planning Poker with example

### What is Planning Poker?

Planning Poker is a consensus-based, gamified estimation technique for story points in Scrum. It leverages the collective wisdom of the team and prevents individual bias from dominating estimates.

### Cards Used:

Modified Fibonacci Sequence: 0, 1, 2, 3, 5, 8, 13, 21, 40, 100, ∞ (infinity), ? (unknown). The gaps between numbers increase at higher values to reflect the uncertainty of larger estimates.

### Step-by-Step Process:

29. The Product Owner presents a User Story: 'As a user, I want to reset my password via email so that I can regain access if I forget it.'
30. Team members ask clarifying questions about scope, edge cases, dependencies.
31. Each developer privately selects a card from their deck.
32. On the count of three, all cards are revealed simultaneously.
33. Results are discussed if there is wide variation.
34. Re-estimation happens until consensus is reached.

### Example:

| Team Member | Card Played | Reasoning |
|---|---|---|
| Developer A | 5 | Standard implementation, familiar with auth module |
| Developer B | 13 | Worried about email delivery reliability, token expiry edge cases |
| QA Engineer | 8 | Needs testing for multiple email providers and edge cases |
| Developer C | 5 | Agrees with Developer A |

Discussion: Developer B explains concerns about email delivery and security. After discussion, the team agrees the complexity warrants 8 story points. Consensus: 8 story points.

### Why Planning Poker is Effective:

- Avoids anchoring bias (all reveal simultaneously).
- Surfaces hidden complexity through discussion.
- Builds shared understanding of user stories.
- Engages all team members equally.

## Q27. Explain: Getting Backlog Item Ready Methods – i) WSJF ii) MoSCoW

### i) Weighted Shortest Job First (WSJF):

WSJF is a prioritization method used in SAFe (Scaled Agile Framework) to sequence backlog items in a way that maximizes economic benefit. It prioritizes items that deliver the most value in the least time.

Formula: WSJF = Cost of Delay / Job Duration

Cost of Delay includes:

- User-Business Value: How much value does this bring to users/business?
- Time Criticality: Is there a deadline or time-sensitive opportunity?
- Risk Reduction / Opportunity Enablement: Does this unlock other work or reduce risk?

Example:

| Item | Business Value | Time Criticality | Risk Reduction | CoD | Job Size | WSJF |
|---|---|---|---|---|---|---|
| Feature A | 5 | 3 | 2 | 10 | 5 | 2.0 |
| Feature B | 8 | 5 | 3 | 16 | 3 | 5.3 |
| Feature C | 2 | 1 | 1 | 4 | 2 | 2.0 |

Feature B has the highest WSJF score (5.3) and should be done first.

### ii) MoSCoW Method:

MoSCoW categorizes requirements into four priority buckets:

- Must Have (M): Non-negotiable requirements. The product cannot be released without these.
- Should Have (S): Important but not critical. The product works without them but is less valuable.
- Could Have (C): Nice-to-have features. Included only if time and budget permit.
- Won't Have (W): Agreed to be excluded from the current scope but may be considered later.

### How it makes backlog items ready:

By categorizing items, the team and stakeholders have a shared understanding of priority before Sprint Planning. Must-Haves are placed at the top of the backlog and refined first.

## Q28. Describe MoSCoW method as a Prioritization Framework for Food Delivery App

### Application of MoSCoW to Food Delivery App:

Consider building a food delivery app like Zomato/Swiggy. Here is how MoSCoW prioritizes the Product Backlog:

### Must Have (M) – Core Features:

- User Registration and Login (Email, Phone, Social Auth)
- Restaurant Listing and Menu Display
- Cart and Order Placement
- Payment Integration (Credit Card, UPI, Wallets)
- Order Tracking (Real-time GPS)
- Push Notifications (Order Confirmed, Out for Delivery)

### Should Have (S) – Important but Not Critical at Launch:

- User Reviews and Ratings for Restaurants
- Coupon and Promo Code System
- Order History and Re-order Functionality
- Multiple Delivery Addresses

### Could Have (C) – Nice-to-Have:

- AI-based Personalized Food Recommendations
- Loyalty Points / Reward Program
- Live Chat with Delivery Partner
- Dietary Filter (Vegan, Gluten-Free, etc.)

### Won't Have (W) – Out of Scope for Now:

- In-app Restaurant Reservation System
- AR/VR Menu Visualization
- Cryptocurrency Payment Option

### Benefit of MoSCoW for this App:

The app can launch with only 'Must Have' features in Sprint 1, gather user feedback, and then add 'Should Have' and 'Could Have' features in subsequent sprints — ensuring the fastest time to market with core value delivered first.

## Q29. Differentiate Slicing Method by Data Variation vs. Slicing Method by Operation (CRUD)

### What is Story Slicing?

Story slicing is the practice of breaking large user stories (epics) into smaller, deliverable stories that can be completed within a single sprint. Two common slicing techniques are by Data Variation and by Operation (CRUD).

### 1. Slicing by Data Variation:

This method slices a feature based on different types of data or user inputs it must handle.

Example: 'As a user, I want to make a payment' sliced by data type:

- Slice 1: Payment by Credit Card
- Slice 2: Payment by Debit Card
- Slice 3: Payment by UPI
- Slice 4: Payment by Net Banking

Each slice handles a different data type but performs the same operation (payment). Teams can deliver payment by Credit Card first and add other methods in subsequent sprints.

### 2. Slicing by Operation (CRUD):

CRUD stands for Create, Read, Update, Delete. This method slices a feature based on the type of operation to be performed on data.

Example: 'As an admin, I want to manage restaurant listings' sliced by operation:

- Slice 1 (Create): Add a new restaurant profile
- Slice 2 (Read): View list of all restaurants
- Slice 3 (Update): Edit restaurant details

- Slice 4 (Delete): Remove a restaurant from the platform

### Comparison Table:

| Parameter | By Data Variation | By Operation (CRUD) |
|---|---|---|
| Basis of slicing | Different types/categories of data | Different actions on the same data |
| Example | Pay by Card, Pay by UPI | Add, View, Edit, Delete restaurant |
| Best for | Features with multiple data categories | Admin panels, management systems |
| Delivery order | Most used data type first | Create → Read → Update → Delete |

## Q30. Explain three important inputs to Sprint Planning meeting (Scrum Master's responsibility)

### The Three Key Inputs to Sprint Planning:

### 1. Product Backlog:

The Product Backlog is the primary input to Sprint Planning. It is the ordered list of all desired features, enhancements, fixes, and technical debt items for the product. The top items (those with the highest priority) must be refined, estimated, and ready before Sprint Planning begins.

Scrum Master's Role: Ensure the PO has groomed and prioritized the top backlog items before the meeting. Ensure all items are clearly defined and estimated.

### 2. Team Capacity:

Team capacity refers to the total available effort (in hours or story points) the team can commit to in the upcoming sprint, after accounting for: vacations and leaves, public holidays, other meetings and non-sprint activities, and individual availability percentages.

Formula: Team Capacity = (Number of Developers) × (Sprint Days) × (Hours/Day) × (Focus Factor)

Scrum Master's Role: Gather and communicate the team's capacity before Sprint Planning so the team doesn't over-commit or under-commit.

### 3. Past Team Velocity:

Velocity is the average number of story points the team completes per sprint over the last 3–5 sprints. It provides a data-driven basis for how much work the team can realistically pull into the new sprint.

Scrum Master's Role: Track velocity over sprints and present historical velocity data to the team during Sprint Planning to anchor capacity decisions. Prevent teams from over-optimistically overloading the sprint.

Example: If average velocity over last 3 sprints = 40 story points, the team should not commit to more than ~40 story points in the new sprint.

## Q31. Justify the total capacity of team with formula and explain with example

### Team Capacity Formula:

Total Team Capacity = Number of Team Members × Sprint Duration (days) × Working Hours per Day × Focus Factor

### Explanation of Variables:

- Number of Team Members: Count of available developers (excluding PO and SM if not coding).
- Sprint Duration: Number of working days in the sprint (e.g., 10 days for a 2-week sprint).
- Working Hours per Day: Typically 8 hours, but Scrum uses 6–7 focused hours to account for overhead.
- Focus Factor: Percentage of time available for sprint work (accounts for meetings, email, admin). Typically 70–80%.

### Detailed Example:

Team: 5 Developers | Sprint: 2 weeks (10 working days) | 8 hrs/day | Focus Factor: 75%

Individual Capacity = 10 days × 8 hours × 0.75 = 60 hours per developer

Total Team Capacity = 5 × 60 = 300 hours

### Adjustment for Leaves/Absence:

Developer A: 1 day leave → loses 8 × 0.75 = 6 hours. Developer B: 0 days leave. Others: 0 days leave.

Adjusted Capacity = 300 - 6 = 294 hours

### Why Capacity Justification Matters:

- Prevents over-commitment: Team does not pull more work than they can handle.
- Prevents under-commitment: Team is not too conservative and wastes capacity.
- Provides a realistic Sprint plan that the team can commit to with confidence.

## Q32. Elaborate Key Activities of Sprint Planning in Detail

### Overview:

Sprint Planning is a time-boxed event (max 8 hours for a 1-month sprint) that kicks off the sprint. The entire Scrum Team attends. The output is a Sprint Backlog and a Sprint Goal.

### Key Activities:

### Activity 1: Set the Sprint Goal

The Product Owner proposes a Sprint Goal — a short statement explaining why the sprint has value. The team collaborates to finalize it. All sprint work should align with this goal.

Example Sprint Goal: 'Enable users to browse and add products to cart.'

### Activity 2: Review Team Capacity

The Scrum Master shares capacity data. The team identifies who is available, accounts for leaves, and determines total hours/story points available for the sprint.

### Activity 3: Select Product Backlog Items (PBIs)

The team reviews the top of the Product Backlog. Items that are estimated, refined, and align with the Sprint Goal are selected. The team selects items until capacity is filled.

### Activity 4: Decompose Selected Items into Tasks

Each selected PBI is broken down into specific technical tasks. Each task should ideally be completable in 1 day or less. This gives the team a clear execution plan.

Example: User Story: 'Add to Cart' → Tasks: Design cart UI, API endpoint for cart, cart DB schema, unit tests for cart logic, integration testing.

### Activity 5: Estimate Tasks and Confirm Commitment

The team verifies that the total estimated effort for all tasks fits within the capacity. The Sprint Backlog is finalized. The team commits to the Sprint Goal.

### Activity 6: Identify Dependencies and Risks

Any cross-team dependencies, technical risks, or blocking issues are surfaced and noted so they can be proactively managed during the sprint.

## UNIT 4: Sprint Burndown Charts & Calculations

### Q33. Burndown: 80 hrs, 8-day Sprint. Calculate remaining work, prepare table, draw chart

**Given Data:**

Total Work: 80 hours | Sprint Duration: 8 days

Daily Completed Work: Day1=10, Day2=5, Day3=15, Day4=10, Day5=10, Day6=10, Day7=10, Day8=10

**Ideal Burndown Rate: 80 / 8 = 10 hours/day**

| Day | Work Done (hrs) | Remaining Work (hrs) | Ideal Remaining (hrs) | Status |
|---|---|---|---|---|
| 0 (Start) | 0 | 80 | 80 | — |
| 1 | 10 | 70 | 70 | On Track |
| 2 | 5 | 65 | 60 | Behind (5hrs) |
| 3 | 15 | 50 | 50 | On Track |
| 4 | 10 | 40 | 40 | On Track |
| 5 | 10 | 30 | 30 | On Track |
| 6 | 10 | 20 | 20 | On Track |
| 7 | 10 | 10 | 10 | On Track |
| 8 | 10 | 0 | 0 | Sprint Complete |

**Analysis:**

- Day 2 was the only day the team fell behind (completed only 5 hrs vs ideal 10 hrs).
- Day 3 recovered the deficit by completing 15 hrs (5 hrs extra).
- From Day 3 onwards, the team was back on track and completed sprint on time.
- The sprint was completed successfully with 0 hours remaining at end of Day 8.

### Q34. Burndown: 50 hrs, 5-day Sprint. Determine remaining work and compare with ideal

**Given Data:**

Total Work: 50 hours | Sprint Duration: 5 days

Daily Completed Work: Day1=5, Day2=10, Day3=5, Day4=15, Day5=15

Ideal Burndown Rate: 50 / 5 = 10 hours/day

| Day | Work Done (hrs) | Actual Remaining (hrs) | Ideal Remaining (hrs) | Variance |
|---|---|---|---|---|
| 0 | 0 | 50 | 50 | 0 |
| 1 | 5 | 45 | 40 | -5 (Behind) |
| 2 | 10 | 35 | 30 | -5 (Behind) |

| Day | Work Done (hrs) | Actual Remaining (hrs) | Ideal Remaining (hrs) | Variance |
|---|---|---|---|---|
| 3 | 5 | 30 | 20 | -10 (Behind) |
| 4 | 15 | 15 | 10 | -5 (Behind) |
| 5 | 15 | 0 | 0 | 0 (Done!) |

### Analysis:

- The team was consistently behind the ideal burndown line from Day 1 to Day 4.
- Day 3 was the worst day — only 5 hrs completed vs ideal 10 hrs, creating a 10-hour deficit.
- Day 4 and Day 5 were high-productivity days (15 hrs each), allowing the team to catch up.
- Sprint was completed on time despite being behind for most of the sprint.
- This is a 'back-loaded' burndown pattern — risky because problems are not identified early.

## Q35. Burndown: 40 hrs, 4-day Sprint. Is team ahead or behind?

### Given Data:

Total Work: 40 hours | Sprint Duration: 4 days

Daily Completed Work: Day1=5, Day2=5, Day3=15, Day4=15

Ideal Burndown Rate: 40 / 4 = 10 hours/day

| Day | Work Done (hrs) | Actual Remaining (hrs) | Ideal Remaining (hrs) | Status |
|---|---|---|---|---|
| 0 | 0 | 40 | 40 | — |
| 1 | 5 | 35 | 30 | Behind by 5 hrs |
| 2 | 5 | 30 | 20 | Behind by 10 hrs |
| 3 | 15 | 15 | 10 | Behind by 5 hrs |
| 4 | 15 | 0 | 0 | Sprint Completed |

### Analysis:

- Days 1 & 2: Team was significantly behind schedule (only 5 hrs/day vs ideal 10 hrs/day).
- Day 2 was the worst point — behind by 10 hours (actual remaining: 30, ideal: 20).
- Days 3 & 4: Team accelerated (15 hrs/day) and caught up.
- Sprint completed on time with 0 remaining work.
- Pattern: Initially behind, recovered at the end — indicates slow start and sprint crunch at end. While sprint was completed, this is a warning sign for future sprints.

## Q36. Burndown: 72 hrs, 6-day Sprint. Prepare table and analyze performance

### Given Data:

Total Work: 72 hours | Sprint Duration: 6 days

Daily Completed Work: Day1=12, Day2=8, Day3=10, Day4=12, Day5=15, Day6=15

Ideal Burndown Rate: 72 / 6 = 12 hours/day

| Day | Work Done (hrs) | Actual Remaining (hrs) | Ideal Remaining (hrs) | Variance | Status |
|---|---|---|---|---|---|
| 0 | 0 | 72 | 72 | 0 | Start |
| 1 | 12 | 60 | 60 | 0 | On Track |
| 2 | 8 | 52 | 48 | -4 | Behind |
| 3 | 10 | 42 | 36 | -6 | Behind |
| 4 | 12 | 30 | 24 | -6 | Behind |
| 5 | 15 | 15 | 12 | -3 | Behind |
| 6 | 15 | 0 | 0 | 0 | Done |

### Performance Analysis:

- Day 1: Perfect — exactly on the ideal line.
- Days 2–5: Team was consistently behind ideal. The gap peaked at 6 hours on Days 3 and 4.
- Days 5–6: High productivity days (15 hrs each) bridged the gap.
- Sprint completed on time.
- Recommendation: The team needs more consistent daily output. The reliance on 'catch-up' days at end of sprint is a risk — if those final days were disrupted, the sprint would not complete.

## Q37. Burndown: 100 hrs, 10-day Sprint. Create table, compare actual vs ideal

### Given Data:

Total Work: 100 hours | Sprint Duration: 10 days

Daily Completed: Day1=8, Day2=12, Day3=10, Day4=5, Day5=15, Day6=10, Day7=10, Day8=10, Day9=10, Day10=10

Ideal Burndown Rate: 100 / 10 = 10 hours/day

| Day | Done (hrs) | Actual Remaining | Ideal Remaining | Variance | Status |
|---|---|---|---|---|---|
| 0 | 0 | 100 | 100 | 0 | Start |
| 1 | 8 | 92 | 90 | -2 | Slightly Behind |
| 2 | 12 | 80 | 80 | 0 | On Track |
| 3 | 10 | 70 | 70 | 0 | On Track |
| 4 | 5 | 65 | 60 | -5 | Behind |
| 5 | 15 | 50 | 50 | 0 | On Track |
| 6 | 10 | 40 | 40 | 0 | On Track |
| 7 | 10 | 30 | 30 | 0 | On Track |

| Day | Done (hrs) | Actual Remaining | Ideal Remaining | Variance | Status |
|---|---|---|---|---|---|
| 8 | 10 | 20 | 20 | 0 | On Track |
| 9 | 10 | 10 | 10 | 0 | On Track |
| 10 | 10 | 0 | 0 | 0 | Sprint Complete |

### Analysis:

- Day 4 was the only problematic day (5 hrs vs ideal 10 hrs, 5-hr deficit).
- Day 5 compensated by delivering 15 hrs (5 hrs extra).
- Days 6–10 were perfectly on track.
- Sprint completed exactly on time — a very well-managed sprint overall.
- This burndown shows a healthy, consistent team with one minor disruption that was quickly corrected.

# UNIT 5: User Stories, Sprint Goals & Backlog Items

## Q38. User Story: 'Launch a website that allows sales to customers inside India'

### User Story (Structured Format):

As an Indian customer, I want to browse and purchase products on a website, so that I can conveniently shop online and have items delivered to my doorstep within India.

### Acceptance Criteria:

- Website is accessible from all major Indian cities.
- All prices are displayed in Indian Rupees (INR).
- Payment via UPI, Net Banking, Credit/Debit Cards (Indian banks).
- Delivery available across all serviceable PIN codes in India.
- Checkout supports Indian address format (State, PIN code).

### Sprint Goal:

'Enable the launch of a basic e-commerce website with product catalog, cart, and payment for Indian customers.'

### Sprint Backlog Items:

| Priority | Backlog Item | Story Points |
|---|---|---|
| 1 | Design and develop Homepage with India-specific branding | 5 |
| 2 | Product Catalog with categories and INR pricing | 8 |
| 3 | User Registration and Login (Phone/Email OTP) | 5 |
| 4 | Shopping Cart (Add/Remove/Update items) | 5 |
| 5 | Checkout with Indian address format and PIN code validation | 5 |
| 6 | UPI/Net Banking/Card Payment Gateway Integration | 13 |
| 7 | Order Confirmation Email with Indian languages support | 3 |
| 8 | Basic SEO for Indian search traffic | 3 |

## Q39. User Story: 'Improve Customer Retention Rate by 20% on Website'

### User Story:

As a business stakeholder, I want to improve the website's customer retention rate by 20%, so that we increase repeat purchases and build long-term customer loyalty.

### Acceptance Criteria:

- Personalized product recommendations based on browsing and purchase history.
- Loyalty points awarded for every purchase, redeemable on future orders.
- Automated re-engagement emails sent to customers inactive for 30 days.
- Retention rate measured via analytics dashboard and shows 20% improvement over 3 months.

### Sprint Goal:

'Implement personalization, loyalty rewards, and re-engagement features to drive repeat purchases and increase retention by 20%.'

### Sprint Backlog Items:

| Priority | Backlog Item | Story Points |
|---|---|---|
| 1 | Implement AI-based product recommendation engine | 13 |
| 2 | Design and implement Loyalty Points System | 8 |
| 3 | Automated re-engagement email campaign (trigger-based) | 8 |
| 4 | Customer retention analytics dashboard | 8 |
| 5 | Personalized homepage for returning users | 5 |
| 6 | Push notifications for abandoned cart | 5 |
| 7 | User preference center (notification opt-ins) | 3 |

## Q40. User Story: 'Enable User to Order Home Style, Everyday Meal with 3 Clicks'

### User Story:

As a busy professional, I want to order my favorite home-style meal in 3 clicks or fewer, so that I can get food quickly without navigating complex menus.

### Acceptance Criteria:

- User can complete a full meal order (select meal → confirm → pay) in exactly 3 taps/clicks.
- 'Favorites' or 'Reorder' feature enables one-click access to previous orders.
- Payment is processed via saved payment method (no re-entry needed).
- Order confirmation displayed within 2 seconds.

### Sprint Goal:

'Streamline the ordering experience so that a returning user can complete an everyday meal order in 3 clicks using saved preferences and quick checkout.'

### 3-Click Journey:

35. Click 1: Open app → 'Reorder' section shows favorite meal (Home Style Thali).
36. Click 2: Tap 'Reorder' → confirms selected meal, delivery address, and saved payment.
37. Click 3: Tap 'Place Order' → order confirmed with 30-minute ETA.

### Sprint Backlog Items:

| Priority | Backlog Item | Story Points |
|---|---|---|
| 1 | Build 'Reorder Last Meal' quick-access button on home screen | 5 |
| 2 | Smart order summary screen (meal + address + payment in one view) | 8 |
| 3 | One-click payment with saved UPI/Card | 8 |
| 4 | Order history with 'Favorite' tagging feature | 5 |

| Priority | Backlog Item | Story Points |
|---|---|---|
| 5 | Real-time order confirmation screen with ETA | 3 |
| 6 | A/B testing framework for 3-click vs current flow | 5 |

# UNIT 6: Burndown Charts, Sprint Review & Retrospective

## Q41. Elaborate the features of Burnout Chart: i) Day ii) Story Points Completed iii) Remaining Work

### What is a Sprint Burndown Chart?

A Sprint Burndown Chart is a visual representation of the amount of work remaining in a sprint versus the time left. It helps the team and stakeholders track sprint progress at a glance.

### i) Day (X-Axis):

The horizontal axis of the burndown chart represents each working day of the sprint (from Day 0 to the last day of the sprint). Each data point on the chart corresponds to the state of work at the end of that day. The progression from left to right represents time moving forward.

### ii) Story Points Completed:

Story points completed represents the amount of work finished by the team each day. As work is completed, it is subtracted from the remaining work. The Actual Burndown Line shows the cumulative story points remaining, which decreases (ideally) as more work is completed. If story points are not being completed fast enough, the actual line will sit above the ideal line.

### iii) Remaining Work (Y-Axis):

The vertical axis shows the total remaining work in story points or hours. It starts at the sprint's total capacity (e.g., 80 hours) at Day 0. A perfectly linear decrease is represented by the Ideal Burndown Line. The team's goal is for the Actual Remaining Work line to meet zero by the last day of the sprint.

### Two Lines on the Chart:

- Ideal Burndown Line: A straight diagonal from total work (Day 0) to zero (last day). Assumes work is completed at a perfectly uniform rate.
- Actual Burndown Line: Plotted from real data each day. It is rarely straight — it shows peaks (when little work was completed) and steep drops (high-productivity days).

### Reading the Chart:

- Actual line ABOVE ideal: Team is behind schedule.
- Actual line BELOW ideal: Team is ahead of schedule.
- Flat segments: Days when no work was completed.
- Steep drops: High-productivity days.

## Q42. Justify how burndown helps to calculate 'Day to finish Sprint on time'

### Forecasting Sprint Completion with Burndown:

The Sprint Burndown Chart is not just a reporting tool — it is a forecasting and early warning system. Here is how it helps calculate whether the team will finish on time:

### Method 1: Trend Line Projection

By plotting the actual burndown line, the team can visually project (extrapolate) whether the line will reach zero by the last sprint day. If the slope of the actual line is less steep than the ideal line, the sprint will not finish on time without corrective action.

### Method 2: Daily Burn Rate Calculation

Formula: Remaining Days to Finish = Remaining Work / Daily Average Burn Rate

Example: Remaining Work = 30 hrs. Average daily burn rate over last 3 days = 8 hrs/day. Days to finish = 30 / 8 = 3.75 days. If only 3 days remain in the sprint, the team needs to increase output or reduce scope.

### Method 3: Scope Adjustment

If the burndown clearly shows the team will not finish all sprint items on time, the Scrum Master and team can: de-scope lower-priority items back to the Product Backlog, increase team focus by removing impediments, ask the Product Owner to adjust scope of existing items.

### Daily Scrum Integration:

The burndown chart is reviewed at every Daily Scrum. Any day the actual line is significantly above the ideal line is a trigger for the team to self-organize and adapt their plan.

The chart gives a data-driven, visual, objective measure of whether the sprint is on track — removing subjective status reports ('I think we're fine') and replacing them with facts.

## Q43. Explain the term 'Confirmation of Acceptance Criteria' in detail

### What are Acceptance Criteria?

Acceptance Criteria (AC) are a set of specific, measurable conditions that a user story must satisfy to be considered 'done' and accepted by the Product Owner. They define the boundaries of a user story and clarify exactly what behavior the system must exhibit.

### Format of Acceptance Criteria:

Usually written in the Given-When-Then (Gherkin) format:

- Given: The initial context or precondition.
- When: The action performed by the user.
- Then: The expected outcome.

Example (Password Reset Story):

- Given: A registered user has forgotten their password.
- When: They click 'Forgot Password' and enter their email.
- Then: They receive a password reset email within 2 minutes with a valid 24-hour token.

### Confirmation of Acceptance Criteria:

Confirmation of Acceptance Criteria is the final step in validating a user story. It involves:

38. Developer Confirmation: Developer verifies that all AC have been implemented before marking the story as done.
39. QA Testing: QA Engineer runs test cases derived from each acceptance criterion to verify correct behavior.
40. Product Owner Review: The Product Owner (and sometimes the user/stakeholder) reviews the implemented feature against the acceptance criteria.
41. Acceptance Decision: The Product Owner formally accepts or rejects the story. Rejected stories are returned to the backlog with notes.

### Why Confirmation Matters:

- Ensures the feature does exactly what the user/business needs.
- Prevents subjective interpretation of 'done.'
- Creates a clear contract between the team and the Product Owner.
- Links directly to the Definition of Done (DoD) to guarantee quality.

## Q44. Explain three questions discussed during Sprint Retrospective

### Purpose of Sprint Retrospective:

The Sprint Retrospective is held after the Sprint Review and before the next Sprint Planning. It is a structured meeting for the Scrum Team to reflect on their process and identify improvements for the next sprint.

### The Three Core Retrospective Questions:

### Question 1: What went well? (Keep Doing)

The team identifies practices, behaviors, and tools that contributed positively to the sprint's success. These should be continued and possibly reinforced.

Examples:

- 'Our Daily Scrums were focused and finished in 15 minutes.'
- 'Code reviews were thorough and caught bugs early.'
- 'The team communicated well about blockers.'

### Question 2: What didn't go well? (Stop Doing / Improve)

The team openly identifies problems, inefficiencies, and friction points encountered during the sprint. This is done in a blame-free, constructive environment.

Examples:

- 'Stories were not refined enough before Sprint Planning, causing estimation confusion.'
- 'Too many meetings outside of Scrum events disrupted focus.'
- 'Deployment issues in the last two days caused delays.'

### Question 3: What can we improve? (Action Items / Start Doing)

For each problem identified, the team proposes concrete, actionable improvements to implement in the next sprint.

Examples:

- 'Schedule a mid-sprint backlog grooming session so stories are better refined.'
- 'Block a 2-hour 'focus time' each morning with no external meetings.'
- 'Set up an automated deployment pipeline by Sprint 5 to eliminate manual errors.'

### Output:

The top 2–3 improvement actions are added directly to the next Sprint Backlog as tasks, ensuring retrospective insights are acted upon.

## Q45. Discuss Sprint Review System for Online Library Management System

### What is a Sprint Review?

The Sprint Review is an informal working meeting held at the end of each Sprint to inspect the Increment and adapt the Product Backlog if needed. It is NOT a status meeting — it is a demonstration and collaboration session.

### Sprint Review for Online Library Management System:

### Participants:

- Scrum Team (Product Owner, Scrum Master, Developers)

- Key Stakeholders (Library Director, Librarians, IT Manager, Representative Students)

### Scenario: Sprint 2 Review

Sprint Goal: 'Enable library members to search for books and reserve them online.'

### Agenda and Flow:

42. Sprint Summary (5 min): Scrum Master presents Sprint 2 dates, goal, and team capacity. Product Owner reports: 'Sprint Goal achieved.'
43. Demo of Completed Features (20 min):

- Book Search by Title, Author, Genre — demonstrated live.
- Book Reservation System — member reserves a book, receives confirmation email.
- Member Dashboard showing current reservations and due dates.

44. Stakeholder Feedback (15 min):

- Library Director: 'Can we add a waitlist feature for popular books?'
- Librarian: 'The dashboard needs a return reminder 3 days before due date.'
- Student: 'It would be great to see if a book is available in other branches.'

45. Product Backlog Adaptation (10 min): Product Owner adds feedback as new backlog items and re-prioritizes. Waitlist feature moves to Sprint 3. Multi-branch availability added to backlog.
46. Upcoming Sprint Preview (5 min): Product Owner previews Sprint 3 Goal: 'Enable digital e-book lending and overdue fine management.'

### Key Outcomes:

- Sprint Goal confirmed achieved.
- 3 new backlog items added from stakeholder feedback.
- Product Backlog re-prioritized for Sprint 3.

## Q46. Explain 3 ways teams work during Sprint: i) Preparation & Planning ii) Execution & Collaboration iii) Review & Adaptation

### i) Preparation and Planning:

Before and at the start of the sprint, the team focuses on ensuring they are ready to execute effectively.

- Backlog Refinement: Product Owner and team refine and estimate top backlog items so they are sprint-ready.
- Sprint Planning: Team selects items, defines the Sprint Goal, and decomposes stories into tasks.
- Capacity Planning: Team assesses availability, identifies leaves, and calculates total sprint capacity.
- Risk Assessment: Technical risks, dependencies, and unclear requirements are surfaced and noted.
- Environment Setup: Necessary tools, environments, and access are prepared before coding begins.

### ii) Execution and Collaboration:

During the sprint, the team works together daily to deliver the Sprint Backlog items.

- Daily Scrum: 15-minute sync each day to coordinate work, surface blockers, and realign on the Sprint Goal.
- Pair Programming and Code Reviews: Developers collaborate on code to improve quality and share knowledge.
- Continuous Integration: Code is integrated and tested frequently (multiple times a day) to detect issues early.
- Impediment Resolution: Scrum Master actively removes blockers so the team maintains flow.
- Collaboration Tools: Teams use Jira, Azure DevOps, or similar tools to track task progress transparently.

### iii) Review and Adaptation:

At the end of the sprint, the team inspects their work and improves.

- Sprint Review: Team demonstrates the completed Increment to stakeholders. Feedback is gathered. Product Backlog is updated.
- Sprint Retrospective: Team reflects on what worked, what didn't, and commits to concrete improvements.
- Updating Velocity: Team updates their velocity metric based on completed story points.
- Next Sprint Preparation: Learnings from this sprint inform the planning of the next sprint immediately.

## Q47. Explain the Usage of Following Tools for Agile: i) Azure DevOps ii) ZenHub

### i) Azure DevOps:

Azure DevOps (by Microsoft) is a comprehensive, cloud-based platform that provides a complete set of tools for planning, developing, testing, and delivering software. It is widely used in enterprise Agile/Scrum teams.

Key Features for Agile:

- Azure Boards: Provides a Kanban board, Sprint board, Backlog management, and Burndown charts. Teams can create Epics, User Stories, Tasks, and Bugs and track them through the sprint lifecycle.
- Azure Repos: Git-based source code management with branch policies, pull requests, and code reviews.
- Azure Pipelines: CI/CD pipeline automation for building, testing, and deploying code automatically at the end of each sprint or on each commit.
- Azure Test Plans: Manual and automated testing management — linked directly to user stories for traceability.
- Sprint Management: Teams configure sprint dates, add backlog items to sprints, track capacity, and monitor burndown — all within the tool.
- Integration: Integrates with GitHub, Jira, Slack, and many other tools.

### ii) ZenHub:

ZenHub is an Agile project management tool built directly inside GitHub. It adds a project management layer on top of GitHub's issue tracker, making it ideal for development teams already using GitHub.

Key Features:

- GitHub-Native: No context switching — sprints, boards, and burndowns are all visible inside the GitHub interface.
- Sprint Boards: Kanban-style board showing issues in columns (To Do, In Progress, Review, Done).
- Epics: Group related GitHub issues into Epics for high-level tracking.
- Burndown Charts: Automatically generated from GitHub issue completion data.
- Velocity Tracking: Tracks team velocity across sprints.
- Roadmaps: Multi-sprint roadmap planning linked to GitHub milestones.

### Comparison:

| Feature | Azure DevOps | ZenHub |
|---|---|---|
| Platform | Standalone/Cloud | Inside GitHub |
| Best for | Enterprise teams | GitHub-native teams |
| CI/CD | Built-in (Azure Pipelines) | Relies on GitHub Actions |
| Cost | Paid (Free tier available) | Freemium |

## Q48. How can a team achieve Sustainable Pace?

### What is Sustainable Pace?

Sustainable Pace is an Agile principle that states the team should be able to maintain a constant development speed (velocity) indefinitely. This means the team should not be overworked in any given sprint to the point where burnout, quality decline, or attrition results.

### Principles Behind Sustainable Pace (from Agile Manifesto):

'Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.'

### How Teams Achieve Sustainable Pace:

- Realistic Sprint Planning: Commit to work that matches the team's actual capacity — not what management wants. Use velocity data to guide commitments.
- No Crunch Culture: Avoid expecting the team to consistently work overtime. Overtime in one sprint creates debt (burnout, bugs, lower output next sprint).
- Focus Factor: Apply a realistic focus factor (e.g., 70–80%) so that daily overhead (meetings, email, code reviews) doesn't eat into sprint commitment.
- Reduce Technical Debt: Allocate a portion of each sprint (e.g., 20%) to technical debt reduction. Accumulating debt slows the team down over time.
- Protect the Team: The Scrum Master shields the team from external interruptions, scope creep, and urgent 'fires' that pull developers away from sprint work.
- Regular Retrospectives: Identify signs of fatigue, process inefficiency, or scope creep early and address them before they become chronic issues.
- Limit WIP (Work in Progress): Using Kanban-style WIP limits prevents team members from juggling too many tasks simultaneously, reducing cognitive load.
- Team Wellbeing: Check in on team morale, encourage time off, and create a psychologically safe environment where members can say 'I can't take on more work.'

# UNIT 7: Scrum Roles, Certification & Scaling

## Q49. Explain Role of Product Owner as 'Captain of the Ship'

### The Metaphor:

The Product Owner is often described as the 'Captain of the Ship.' Just as a ship's captain is responsible for setting the course, making critical navigation decisions, and ensuring the ship reaches its destination safely, the Product Owner charts the course for the product and steers it toward maximum business value.

### Responsibilities Mapped to the Metaphor:

- Setting the Course (Product Vision): The captain decides where the ship is going. Similarly, the Product Owner defines and communicates the Product Goal and long-term vision to the team.
- Navigation (Backlog Prioritization): The captain adjusts the route based on weather and obstacles. The Product Owner continuously prioritizes and re-prioritizes the Product Backlog based on market conditions, stakeholder feedback, and business priorities.
- Cargo Management (Maximizing Value): The captain ensures the ship carries the most valuable cargo. The PO ensures the team works on the most valuable features first, maximizing the return on investment.
- Commanding Decisions (Sole Backlog Authority): The captain makes final call on direction. The PO has sole authority over the Product Backlog — no one overrides their ordering decisions.
- Communication with the Crew (Team Alignment): The captain communicates the mission clearly. The PO communicates user stories, acceptance criteria, and the Sprint Goal so the team understands the 'why' behind each feature.
- Responding to Storms (Handling Change): A captain responds to unexpected storms. The PO responds to sudden business changes, regulatory requirements, or market shifts by updating the backlog immediately.

### Key Accountability:

The Product Owner is accountable to the organization that the product delivers maximum value. Like a captain who cannot blame the crew for arriving at the wrong port, the PO owns the product's success or failure in terms of business value.

## Q50. Explain Role of Scrum Master as 'Servant Leader'

### What is a Servant Leader?

Servant Leadership is a leadership philosophy where the leader's primary role is to serve others — removing obstacles, enabling growth, and prioritizing the team's needs above their own authority. The Scrum Master is the ultimate servant leader in the Scrum framework.

### Characteristics of the Scrum Master as Servant Leader:

- Serving the Development Team: The SM listens actively to the team's concerns, helps remove impediments, and ensures the team has everything it needs to do its best work. The SM does not give orders — they empower.
- Serving the Product Owner: The SM helps the PO understand Scrum, facilitates backlog grooming, and assists in communicating effectively with the development team.
- Serving the Organization: The SM coaches the organization in Agile adoption, removes systemic impediments (organizational policies that hinder the team), and builds Agile capability across teams.

### Specific Servant Leader Actions:

47. Facilitating Scrum Events: Ensures Sprint Planning, Daily Scrum, Sprint Review, and Retrospective are effective and time-boxed.

48. Removing Impediments: Actively identifies and removes blockers (technical, organizational, or interpersonal) that slow the team.
49. Coaching: Coaches team members in Agile practices, Scrum values, and self-organization.
50. Shielding the Team: Protects the team from external disruptions, scope creep, and unreasonable demands.
51. Building Trust: Creates a safe environment where team members can speak openly in retrospectives without fear.

### What the Scrum Master is NOT:

- Not a Project Manager: SM does not assign tasks or manage timelines.
- Not a Team Lead: SM has no authority over team members.
- Not a Status Reporter: SM does not report team performance to management.

## Q51. Explain Role of Scrum Development Team as 'Group with Diverse Skills'

### What Makes the Development Team a 'Group with Diverse Skills'?

The Scrum Development Team (Developers in modern Scrum Guide) is a cross-functional group that collectively possesses all the skills needed to deliver a working product Increment at the end of every Sprint. No single team member has all the skills, but together, the team does.

### Types of Skills in a Diverse Development Team:

| Skill Area | Role | Contribution |
|---|---|---|
| Frontend Dev | UI Developer | Builds user-facing interfaces |
| Backend Dev | API/Server Developer | Business logic, databases, APIs |
| QA Engineer | Tester | Ensures quality, writes test cases |
| DevOps Engineer | Infrastructure | CI/CD, deployment, monitoring |
| UX Designer | Design | User research, wireframes, UX flows |
| Data Engineer | Analytics | Reporting, data pipelines |

### Why Diverse Skills Matter in Scrum:

- Self-Sufficiency: A diverse team does not need to wait for external experts. All skills needed to complete a user story are within the team.
- End-to-End Delivery: A story goes from design → code → test → deploy within the same team in one sprint — no handoffs across departments.
- Collective Ownership: The team collectively owns the code and the quality, not any individual. 'Our code' vs 'my code.'
- Peer Learning: Diverse skills enable knowledge sharing. Developers learn testing; testers learn about system architecture.
- Resilience: If one team member is absent, others with overlapping or adjacent skills can step in, preventing sprint failure.

### Size Recommendation:

The Scrum Team (including PO and SM) should ideally be 5–11 people. Small enough to be agile and self-organizing; large enough to have the diverse skills needed.

## Q52. Describe Scrum Project Community with 'Plan-Do-Check-Act' (PDCA) Activity

### What is PDCA?

The Plan-Do-Check-Act (PDCA) cycle (also known as the Deming Cycle) is a continuous improvement framework used in quality management. Scrum naturally embodies the PDCA cycle in every sprint.

### PDCA Mapped to Scrum:

### PLAN → Sprint Planning

The team plans what to do in the upcoming sprint. They define the Sprint Goal, select Product Backlog Items, and decompose them into tasks. The Sprint Backlog is the plan.

- Activity: Define sprint goal, estimate stories, assign tasks, identify risks.

### DO → Sprint Execution

The team executes the plan during the sprint. Developers code, test, and integrate. Daily Scrums ensure daily coordination and micro-adjustments.

- Activity: Write code, run unit tests, do code reviews, deploy to staging.

### CHECK → Sprint Review

At the end of the sprint, the team and stakeholders inspect the Increment against the Sprint Goal and acceptance criteria. Did the team deliver what was planned? Does it work as expected?

- Activity: Demo features to stakeholders, gather feedback, update Product Backlog.

### ACT → Sprint Retrospective

The team reflects on the process and identifies improvements. Concrete actions are taken to improve quality, efficiency, and team dynamics in the next sprint. The cycle then repeats.

- Activity: Identify what went well/badly, create improvement action items, update working agreements.

### The Continuous Loop:

Each sprint is one full PDCA cycle. The act of one sprint feeds directly into the plan of the next. Over many sprints, this cycle drives continuous improvement in both the product and the team's process — which is the essence of Agile.

## Q53. Elaborate scrum.org for Assessment and Certification of Different Roles

### What is Scrum.org?

Scrum.org is the organization founded by Ken Schwaber (co-creator of Scrum) that provides education, training, and professional certifications for Scrum practitioners worldwide. It maintains the official Scrum Guide and offers rigorous, industry-recognized certification programs.

### Key Certifications for Each Role:

### 1. Professional Scrum Master (PSM):

- PSM I: Foundational understanding of Scrum framework, roles, events, and artifacts. Multiple-choice exam.
- PSM II: Advanced Scrum Master knowledge — facilitation, coaching, conflict resolution. Essay-style exam.
- PSM III: Expert-level Scrum Mastery — requires demonstration of professional maturity and deep facilitation skills.

### 2. Professional Scrum Product Owner (PSPO):

- PSPO I: Understanding of the Product Owner role, product value, and backlog management.
- PSPO II: Advanced product management skills — stakeholder management, product strategy.
- PSPO III: Expert-level product ownership and organizational transformation.

### 3. Professional Scrum Developer (PSD):

- PSD I: Technical Scrum practices — coding in a Scrum context, CI/CD, TDD, and refactoring within sprints.

### 4. Other Certifications:

- Professional Agile Leadership (PAL): For leaders and managers supporting Agile teams.
- Scaled Professional Scrum (SPS): For scaling Scrum with Nexus framework.
- Professional Scrum with Kanban (PSK): Combining Scrum with Kanban practices.

### Assessment Model:

- Online, time-boxed assessments (60–90 min). No prerequisites for most exams. Passing score: 85%+ for PSM I, PSPO I.
- Certifications do not expire and are globally recognized.
- Free Scrum Open assessment available for practice at scrum.org.

## Q54. Explain 'Scaled Agile Framework Parameters': 1. Team of Teams 2. Program Increment

### What is SAFe (Scaled Agile Framework)?

SAFe is a framework for scaling Agile practices to large enterprises with multiple teams working on the same product or portfolio. It provides structured guidance for aligning strategy, portfolio, program, and team-level Agile execution.

### 1. Team of Teams:

In SAFe, an Agile Release Train (ART) is the primary organizational construct — it is literally a 'Team of Teams.' An ART consists of 5–12 Agile teams (50–125 people) aligned to a common mission, product, or value stream.

Key Characteristics:

- Each team within the ART is a standard Scrum team with a PO, SM, and developers.
- All teams in the ART run synchronized sprints (called Iterations in SAFe).
- A Program Backlog is maintained at the ART level, managed by the Product Manager (above POs).
- System Architect/Engineers provide technical guidance across all teams.
- Release Train Engineer (RTE): The ART-level Scrum Master who facilitates cross-team ceremonies and removes systemic impediments.

Benefits:

- Teams can work on different parts of a large system simultaneously.
- Cross-team dependencies are managed through the PI Planning event.
- Alignment ensures all teams move toward the same Program Increment objective.

### 2. Program Increment (PI):

A Program Increment is the equivalent of a Scrum Sprint, but at the ART (multi-team) level. A PI is typically 8–12 weeks long and consists of 4–5 Iterations (sprints) plus one Innovation & Planning (IP) Iteration.

PI Planning:

- PI Planning is a 2-day, face-to-face (or virtual) event held at the start of every PI.
- All members of the ART (all teams) attend.

- Business owners present priorities and vision.
- Teams plan their iterations for the upcoming PI, identify cross-team dependencies, and commit to PI Objectives.

PI Objectives:

- Each team defines 4–5 PI Objectives — business-value-oriented goals for the PI.
- Business value is scored by business owners (1–10 scale).
- At the end of the PI, actual vs. planned business value is measured.

IP Iteration (Innovation & Planning Iteration):

- Used for: PI Planning preparation, innovation (hackathons), Inspect & Adapt workshop, and addressing any unfinished work.
- Not used for adding new features — it is a buffer and planning period.

### Why SAFe Matters:

SAFe enables large organizations (banks, telecoms, insurance companies) with hundreds of developers to remain Agile — delivering value frequently, aligning to business strategy, and continuously improving — all while managing the complexity of multiple teams working on interrelated systems.